Experiment Report

(Experiment 3: MATLAB Simulation of Expectation, Variance and Covariance)

Class:

Name:

Student ID:

Course Name: 概率和数理统计

Instructor:

Date: 2026.6.17

Experiment 3: MATLAB Simulation of Expectation, Variance and Covariance

I、 Experiment Objectives

1. To understand the basic concepts of expectation, variance, and covariance through MATLAB simulation.
2. To learn how to generate random samples and compute statistical quantities using both manual formulas and MATLAB built-in functions.
3. To understand how sample size and bin size affect the approximation of a probability distribution and expected value.
4. To observe the behavior of random variables in special cases, including the St. Petersburg paradox.
5. To analyze the dynamic relationship between time and risk through a 1D Random Walk simulation, demonstrating how variance (uncertainty) strictly expands over time despite a constant, zero expected value.

II、 Experiment Preview

Before attending the lab, students should review and familiarize themselves with the following key concepts:

1. The definition and calculation of expected value (mean).
2. The definition and calculation of variance.
3. The meaning and calculation of covariance.
4. The use of **normal distribution** in random sampling and simulation.
5. The relationship between histograms, probability density functions (PDF), and Riemann sums.
6. The basic properties of **expectation** and **variance** for linear combinations of random variables.
7. The basic idea of the **St. Petersburg paradox** and why it is important in probability theory.

8. Basic MATLAB operations, including random number generation, loops, plotting, and built-in statistical functions.

III、 Experiment Content

1. Task 1: Generate random samples from a standard normal distribution, and calculate the sample mean and variance using both manual formulas and MATLAB built-in functions.
2. Task 2: Simulate data from a normal distribution, construct histograms with different sample sizes and bin numbers, and use the Riemann sum to approximate the expected value.
3. Task 3: Generate correlated random variables, calculate covariance, and verify the properties of expectation and variance for linear combinations of random variables.
4. Task 4: Simulate the St. Petersburg game, compute the cumulative sample mean and variance, and observe their changes as the number of games increases.
5. Task 5: Simulate a 1D Random Walk to represent a "fair" casino game, calculate the group mean and variance at each time step, and visualize how individual risk (variance) expands infinitely over time despite a constant zero mean.

IV、 Experiment Tasks

(Students should complete all tasks according to the experiment manual and provide detailed experimental procedures and results. Each task may include the following parts: source code, descriptions of the code, and a screenshot of the terminal output (result).)

1. Task 1 - MATLAB Basics and The Calculation of The Sample Mean
    1) Source code:

```
% ============================================================
% Experiment 3 - Task 1
% Student Fill-in Template: Mean and Variance
% ============================================================
% Fill in every ______ before running this script.

% --- Initialization ---
clear;
clc;
close all;

% --- Step 1: Generate Random Data ---
% Generate N samples from the standard normal distribution.
```

```
N = 100000;
X = randn(1, N);        % Hint: use randn(1, N)

% --- Step 2: Calculate Mean Manually ---
% Formula: mean = sum(x_i) / N
mean_math = sum(X) / N;

% --- Step 3: Calculate Variance Manually ---
% Formula for sample variance:
% var = sum((x_i - mean)^2) / (N - 1)
% Hint: use .^2, not ^2, because X is an array.
var_math = sum((X - mean_math) .^ 2) / (N-1);

% --- Step 4: Use MATLAB Built-in Functions ---
mean_builtin = mean(X);    % Hint: mean(X)
var_builtin = var(X);     % Hint: var(X)

% --- Step 5: Display and Compare Results ---
disp('--- Mean of X ---');
fprintf('Manual Formula : %.4f\n', mean_math);
fprintf('MATLAB Built-in: %.4f\n\n', mean_builtin);

disp('--- Variance of X ---');
fprintf('Manual Formula : %.4f\n', var_math);
fprintf('MATLAB Built-in: %.4f\n', var_builtin);

% Question for report:
% Are mean_math and mean_builtin close to each other?
% Are var_math and var_builtin close to each other?
```

2) Description of the code:

① Generate random data.

```
N = 100000;
X = randn(1, N);        % Hint: use randn(1, N)
```

② Calculate mean and variance manually.

```
mean_math = sum(X) / N;
var_math = sum((X - mean_math) .^ 2) / (N-1);
```

③ Calculate mean and variance using MATLAB built-in functions.

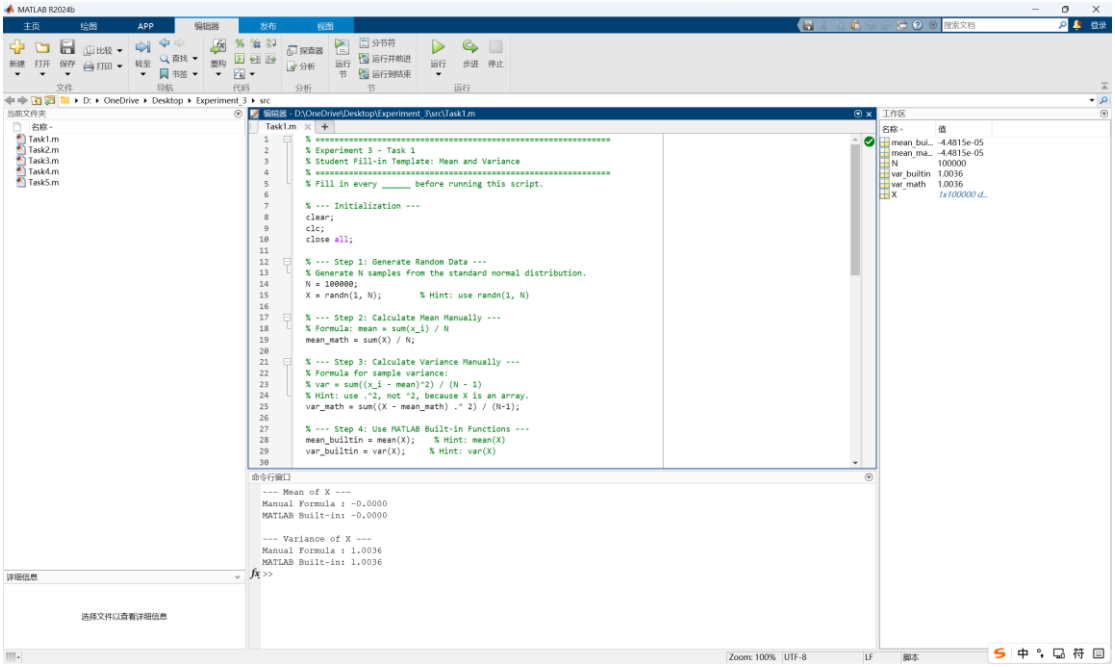
```
mean_builtin = mean(X);    % Hint: mean(X)
var_builtin = var(X);     % Hint: var(X)
```

④ Display and compare results.

```
disp('--- Mean of X ---');
fprintf('Manual Formula : %.4f\n', mean_math);
fprintf('MATLAB Built-in: %.4f\n\n', mean_builtin);

disp('--- Variance of X ---');
fprintf('Manual Formula : %.4f\n', var_math);
fprintf('MATLAB Built-in: %.4f\n', var_builtin);
```

3) Screenshot of the terminal output:



2. Task 2 - Modeling The Probability Distribution

1) Source code:

```
% ================================================================
% Experiment 3 - Task 2
% Student Fill-in Template: Probability Distribution and Riemann Sum
% ================================================================
% Fill in every ______ before running this script.

% --- Initialization ---
clear;
clc;
close all;

% --- Theoretical Parameters ---
% The target population is a normal distribution with mean 10
% and standard deviation 2.
mu_true = 10;
sigma_true = 2;

% Compare different sample sizes and different numbers of bins.
```

```
N_values = [100000, 1000000];       % Hint: 100000 and 1000000
Bin_values = [10, 100];     % Hint: 10 and 100

figure;
plot_idx = 1;

% --- Outer Loop: Change Sample Size ---
for i = 1:length(N_values)
    N = N_values(i);

    % Generate normal-distribution data.
    % Formula: data = mu + sigma * standard_normal_samples
    data = mu_true + sigma_true * randn(1,N);

    % Calculate actual sample mean.
    actual_sample_mean = mean(data);

    fprintf('\n================================================\n');
    fprintf(' EXPERIMENT GROUP: Sample Size N = %d\n', N);
    fprintf('================================================\n');
    fprintf('Theoretical Mean     : %.6f\n', mu_true);
    fprintf('Actual Sample Mean   : %.6f\n\n', actual_sample_mean);

    % --- Inner Loop: Change Number of Bins ---
    for j = 1:length(Bin_values)
        num_bins = Bin_values(j);

        % Step 1: Draw probability density histogram.
        subplot(2, 2, plot_idx);
        h = histogram(data, num_bins, 'Normalization', 'pdf');
        title(sprintf('N = %d, Bins = %d', N, num_bins));
        xlabel('X Value');
        ylabel('Probability Density f(x)');
        plot_idx = plot_idx + 1;

        % Step 2: Extract histogram information.
        % dx is the width of each bar.
        dx = h.BinWidth;

        % centers are the center x-coordinates of the bars.
        centers = (h.BinEdges(1:end-1) + h.BinEdges(2:end)) / 2;

        % areas are the probability areas of all bars.
        % Since h.Values are density heights, area = height * width.
        areas = h.Values * dx;

        % Step 3: Approximate expected value by Riemann sum.
        % Formula: E(X) approximately equals sum(center * area)
        riemann_sum = sum(centers .* areas);
```

```
        % Step 4: Calculate error from actual sample mean.
        error_from_sample = riemann_sum - actual_sample_mean;
        error_from_theory = riemann_sum - mu_true;

        fprintf('  --- Bins = %d (Width dx = %.4f) ---\n', num_bins,
dx);
        fprintf('  Riemann Sum Result      : %.6f\n', riemann_sum);
        fprintf('  Error from Sample Mean   : %.6f\n',
error_from_sample);
        fprintf('  Error from Theory Mean   : %.6f\n\n',
error_from_theory);
    end
 end

 % Questions for report:
 % 1. Does a larger N usually make the sample mean closer to mu_true?
 % 2. How does changing the number of bins affect the Riemann sum
result?
```

2) Description of the code:

① Define the theoretical parameters and Compare different sample sizes and different numbers of bins.

```
mu_true = 10;
sigma_true = 2;

N_values = [100000, 1000000];       % Hint: 100000 and 1000000
Bin_values = [10, 100];     % Hint: 10 and 100
```

② Outer loop over different sample sizes.

```
for i = 1:length(N_values)
    N = N_values(i);
```

③ Generates normally distributed random data and calculate the actual sample mean.

```
    data = mu_true + sigma_true * randn(1,N);
    actual_sample_mean = mean(data);
```

④ Prints information about the current experiment group to the Command Window: the sample size N, the theoretical mean (6 decimal places), and the actual sample mean.

```
    fprintf('\n================================================\n');
    fprintf(' EXPERIMENT GROUP: Sample Size N = %d\n', N);
```

```
    fprintf('================================================\n');
    fprintf('Theoretical Mean     : %.6f\n', mu_true);
    fprintf('Actual Sample Mean   : %.6f\n\n', actual_sample_mean);
```

⑤ Inner loop over different numbers of bins.

```
    for j = 1:length(Bin_values)
        num_bins = Bin_values(j);
```

⑥ Draw probability density histogram.

```
        subplot(2, 2, plot_idx);
        h = histogram(data, num_bins, 'Normalization', 'pdf');
        title(sprintf('N = %d, Bins = %d', N, num_bins));
        xlabel('X Value');
        ylabel('Probability Density f(x)');
```

⑦ Extract histogram information.

```
        dx = h.BinWidth;
        centers = (h.BinEdges(1:end-1) + h.BinEdges(2:end)) / 2;
        areas = h.Values * dx;
```

⑧ Approximate expected value by Riemann sum.

```
        riemann_sum = sum(centers .* areas);
```

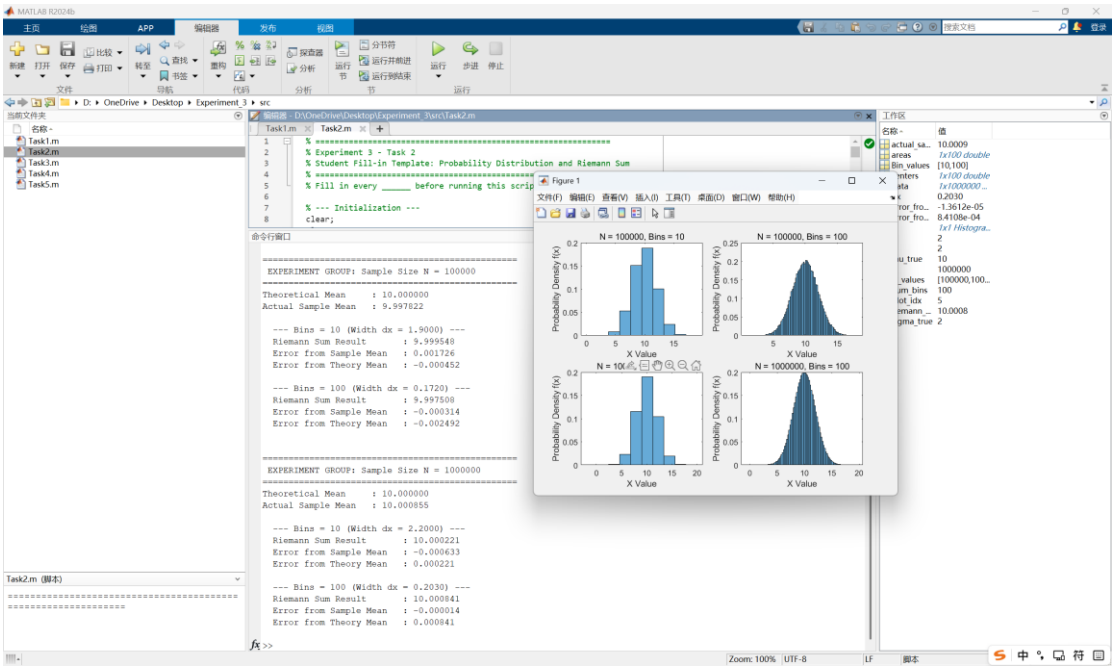
⑨ Calculate error from actual sample mean.

```
        error_from_sample = riemann_sum - actual_sample_mean;
        error_from_theory = riemann_sum - mu_true;
```

⑩ Display the current bin count, bin width, Riemann sum result, and both errors, and close the inner and outer loops.

```
        fprintf('  --- Bins = %d (Width dx = %.4f) ---\n', num_bins,
dx);
        fprintf('  Riemann Sum Result      : %.6f\n', riemann_sum);
        fprintf('  Error from Sample Mean   : %.6f\n',
error_from_sample);
        fprintf('  Error from Theory Mean   : %.6f\n\n',
error_from_theory);
    end
 end
```

3) Screenshot the of terminal output:



3. Task 3 - Covariance and Linear Combinations of Random Variables

1) Source code:

```
% ================================================================
% Experiment 3 - Task 3
% Student Fill-in Template: Covariance and Linear Combinations
% ================================================================
% Fill in every ______ before running this script.

% --- Initialization ---
clear;
clc;
close all;

% --- Step 1: Generate Correlated Data ---
N = 10000;

% Generate two independent standard normal variables.
X = randn(1, N);
Z = randn(1, N);

% Create Y by mixing X and Z.
% This makes Y correlated with X.
Y = 0.8 * X + 0.2 * Z;       % Hint: 0.8*X + 0.2*Z

% Create W = aX + bY.
a = 2;
b = 3;
W = a * X + b * Y;
```

```
% --- Step 2: Extract Covariance ---
% cov(X, Y) returns a 2-by-2 covariance matrix.
cov_matrix = cov(X, Y);
cov_builtin = cov_matrix(1, 2);     % Hint: row 1, column 2

% --- Step 3: Verify Mean Property ---
% Formula: E(W) = aE(X) + bE(Y)
actual_mean_W = mean(W);
theory_mean_W = a * mean(X) + b * mean(Y);

% --- Step 4: Verify Variance Property ---
% Actual variance from data:
actual_var_W = var(W);

var_X = var(X);
var_Y = var(Y);

% Correct formula:
% Var(aX + bY) = a^2 Var(X) + b^2 Var(Y) + 2ab Cov(X,Y)
var_correct = a ^ 2 * var_X + b ^ 2 * var_Y + 2 * a * b * cov_builtin;

% Wrong formula:
% This ignores covariance and should not match well when X and Y are
related.
var_wrong = a ^ 2 * var_X + b ^ 2 * var_Y;

% --- Step 5: Display Results ---
disp('--- 1. Mean Property Verification ---');
fprintf('Actual Mean E(W)     : %.4f\n', actual_mean_W);
fprintf('Theory aE(X) + bE(Y) : %.4f\n\n', theory_mean_W);

disp('--- 2. Variance Property Verification ---');
fprintf('Covariance Cov(X,Y)       : %.4f\n', cov_builtin);
fprintf('Actual Variance Var(W)    : %.4f\n', actual_var_W);
fprintf('Correct Formula (With Cov): %.4f\n', var_correct);
fprintf('Wrong Formula (No Cov)    : %.4f\n\n', var_wrong);

disp('--- Error Analysis ---');
fprintf('Error caused by ignoring Covariance: %.4f\n',
abs(actual_var_W - var_wrong));

% Question for report:
% Why must covariance be included when X and Y are correlated?
```

2) Description of the code:

① Generate correlated data.

```
N = 10000;
```

```
% Generate two independent standard normal variables.
X = randn(1, N);
Z = randn(1, N);

% Create Y by mixing X and Z.
% This makes Y correlated with X.
Y = 0.8 * X + 0.2 * Z;      % Hint: 0.8*X + 0.2*Z

% Create W = aX + bY.
a = 2;
b = 3;
W = a * X + b * Y;
```

② Extract covariance.

```
cov_matrix = cov(X, Y);
cov_builtin = cov_matrix(1, 2);     % Hint: row 1, column 2
```

③ Verify mean property.

```
actual_mean_W = mean(W);
theory_mean_W = a * mean(X) + b * mean(Y);
```

④ Verify variance property.

```
actual_var_W = var(W);

var_X = var(X);
var_Y = var(Y);

% Correct formula:
% Var(aX + bY) = a^2 Var(X) + b^2 Var(Y) + 2ab Cov(X,Y)
var_correct = a ^ 2 * var_X + b ^ 2 * var_Y + 2 * a * b * cov_builtin;

% Wrong formula:
% This ignores covariance and should not match well when X and Y are
related.
var_wrong = a ^ 2 * var_X + b ^ 2 * var_Y;
```

⑤ Display results.

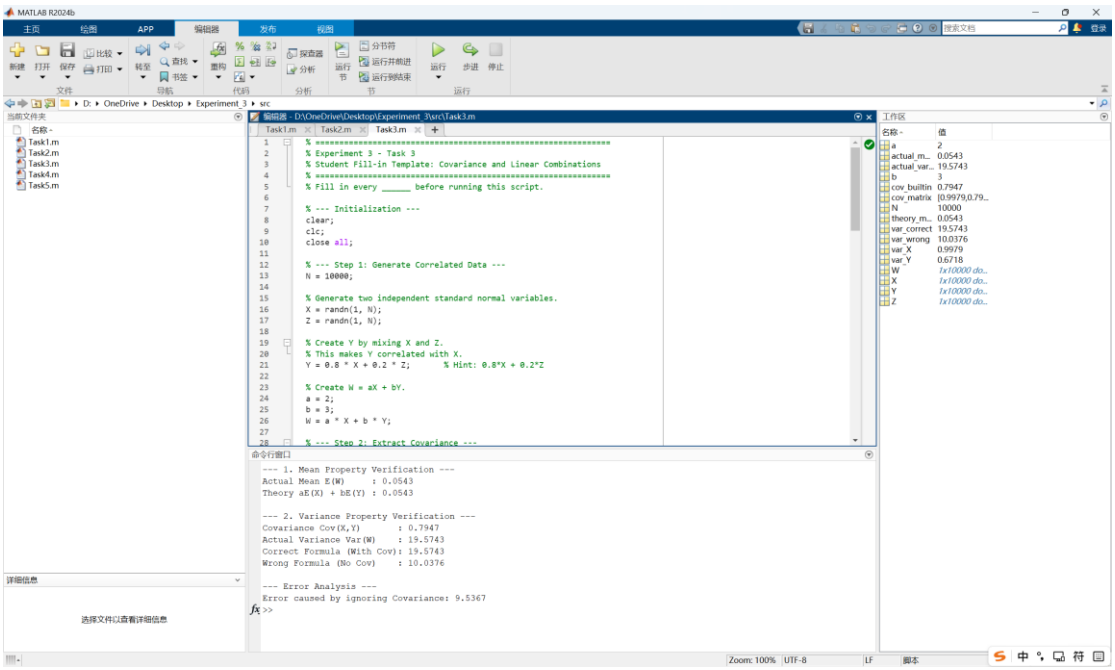
```
disp('--- 1. Mean Property Verification ---');
fprintf('Actual Mean E(W)     : %.4f\n', actual_mean_W);
fprintf('Theory aE(X) + bE(Y) : %.4f\n\n', theory_mean_W);

disp('--- 2. Variance Property Verification ---');
fprintf('Covariance Cov(X,Y)      : %.4f\n', cov_builtin);
```

```
 fprintf('Actual Variance Var(W)    : %.4f\n', actual_var_W);
 fprintf('Correct Formula (With Cov): %.4f\n', var_correct);
 fprintf('Wrong Formula (No Cov)    : %.4f\n\n', var_wrong);

 disp('--- Error Analysis ---');
 fprintf('Error caused by ignoring Covariance: %.4f\n',
abs(actual_var_W - var_wrong));
```

3) Screenshot of the terminal output:



4. Task 4 - The St. Petersburg Paradox (Expectation and Variance at the Limit)

1) Source code:

```
% ================================================================
% Experiment 3 - Task 4
% Student Fill-in Template: The St. Petersburg Paradox
% ================================================================
% Fill in every ______ before running this script.

% --- Initialization ---
clear;
clc;
close all;

% --- Step 1: Simulate the Game ---
% Repeat the game many times.
N = 100000;                % Hint: 100000
payoffs = zeros(1, N);          % Hint: zeros(1, N)

for i = 1:N
    % k is the number of flips until the first Head appears.
```

```
    k = 1;

    % rand() < 0.5 represents Tails.
    % Keep flipping while the result is Tails.
    while rand() < 0.5
        k = k + 1;
    end

    % If the first Head appears on the k-th flip, payoff is 2^k.
    payoffs(i) = 2 ^ k;
end

% --- Step 2: Calculate Cumulative Statistics ---
n_idx = 1:N;            % Hint: 1:N

% Cumulative mean: average payoff up to game i.
cum_mean = cumsum(payoffs) ./ n_idx;

% Cumulative variance:
% Use Var(X) = E(X^2) - [E(X)]^2.
cum_var = cumsum(payoffs .^ 2) ./ n_idx - cum_mean .^ 2;

% --- Step 3: Print Results ---
fprintf('--- Results after %d Games ---\n', N);
fprintf('Final Sample Mean    : %.2f\n', cum_mean(end));
fprintf('Final Sample Variance: %.2f\n', cum_var(end));
fprintf('Highest Prize Won    : %d\n', max(payoffs));

% --- Step 4: Visualize the Paradox ---
figure;

subplot(2, 1, 1);
plot(n_idx, cum_mean, 'LineWidth', 1.5);
title('Cumulative Sample Mean');
xlabel('Number of Games');
ylabel('Average Profit ($)');
grid on;

subplot(2, 1, 2);
plot(n_idx, cum_var, 'r', 'LineWidth', 1.5);
title('Cumulative Sample Variance');
xlabel('Number of Games');
ylabel('Variance');
grid on;

% Question for report:
% If a casino asks you to pay $20 to play this game once,
% would you accept? Use your final sample mean to explain.
```

2) Description of the code:

① Set the number of game repetitions to N = 100000. Pre-allocate a 1-by-N vector payoffs filled with zeros to store the reward of each game.

```
N = 100000;                % Hint: 100000
payoffs = zeros(1, N);          % Hint: zeros(1, N)
```

② Loop over N games. As long as the random number is less than 0.5 (representing Tails, 50% chance), keep flipping and increment k. When the loop exits (a Head appears), the payoff is 2^k, stored in payoffs(i).

```
for i = 1:N
    % k is the number of flips until the first Head appears.
    k = 1;

    % rand() < 0.5 represents Tails.
    % Keep flipping while the result is Tails.
    while rand() < 0.5
        k = k + 1;
    end

    % If the first Head appears on the k-th flip, payoff is 2^k.
    payoffs(i) = 2 ^ k;
end
```

③ Creates the vector n_idx = 1:N, representing the game indices.

```
n_idx = 1:N;            % Hint: 1:N
```

④ Computes the cumulative sample mean.

```
cum_mean = cumsum(payoffs) ./ n_idx;
```

⑤ Computes the cumulative sample variance using the identity $Var(X) = E(X^2) - [E(X)]^2$.

```
cum_var = cumsum(payoffs .^ 2) ./ n_idx - cum_mean .^ 2;
```

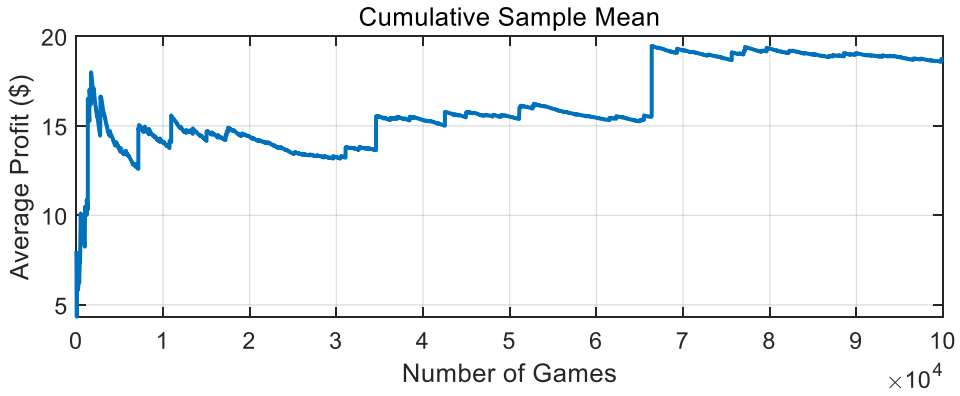
⑥ Prints simulation results: the final sample mean, final sample variance, and the highest prize won in any single game.

```
fprintf('--- Results after %d Games ---\n', N);
fprintf('Final Sample Mean    : %.2f\n', cum_mean(end));
```

```
fprintf('Final Sample Variance: %.2f\n', cum_var(end));
fprintf('Highest Prize Won    : %d\n', max(payoffs));
```

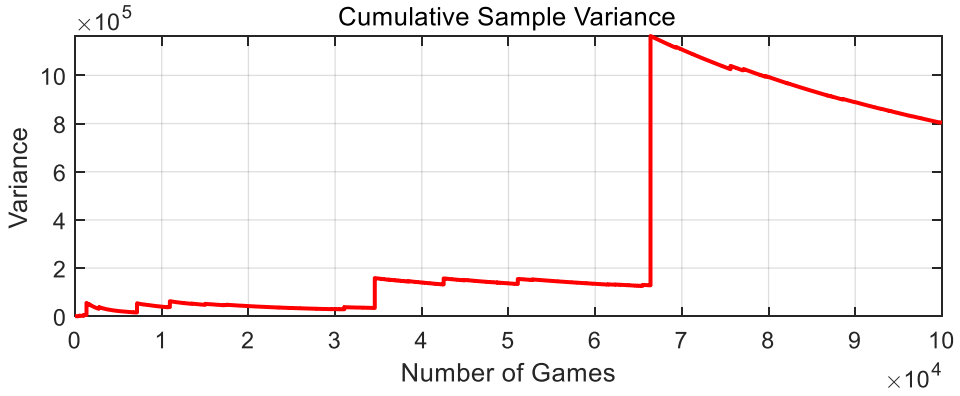
⑦ Plots the cumulative sample mean.

```
subplot(2, 1, 1);
plot(n_idx, cum_mean, 'LineWidth', 1.5);
title('Cumulative Sample Mean');
xlabel('Number of Games');
ylabel('Average Profit ($)');
grid on;
```

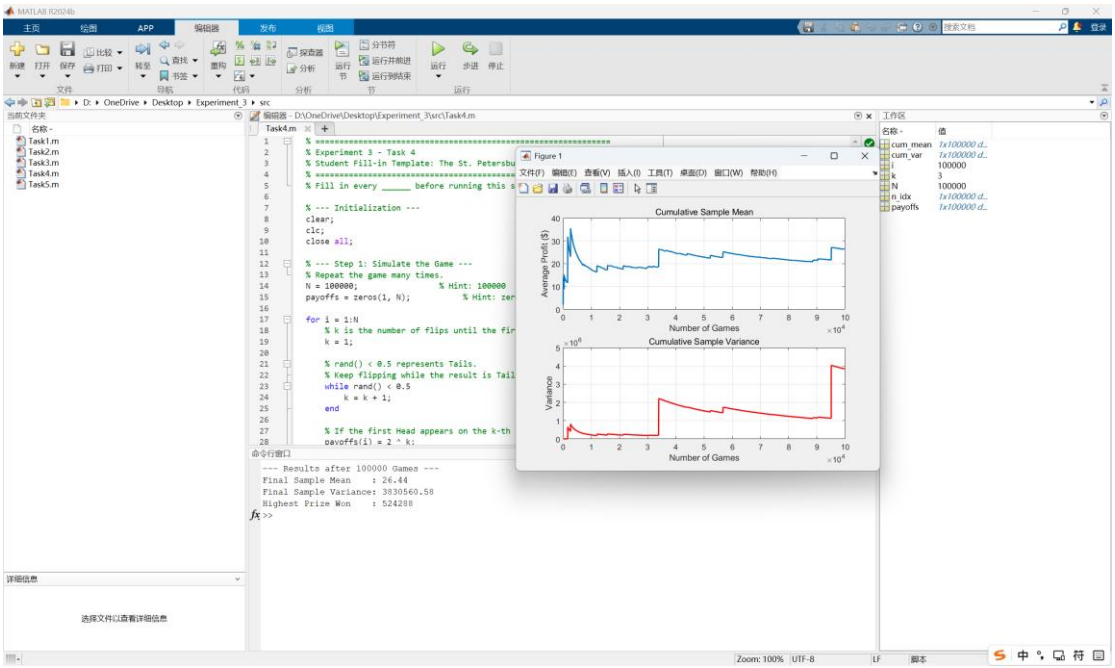


⑧ Plots the cumulative sample variance.

```
subplot(2, 1, 2);
plot(n_idx, cum_var, 'r', 'LineWidth', 1.5);
title('Cumulative Sample Variance');
xlabel('Number of Games');
ylabel('Variance');
grid on;
```



3) Screenshot of the terminal output:



5. Task 5 - Expectation and Variance at the Limit The 1D Random Walk (Time, Risk, and Expanding Variance)

1) Source code:

```
% ================================================================
% Experiment 3 - Task 5
% Student Fill-in Template: 1D Random Walk
% ================================================================
% Fill in every ______ before running this script.

% --- Initialization ---
clear;
clc;
close all;

% --- Step 1: Setup Simulation Parameters ---
num_people = 1000;        % Hint: 1000
num_steps = 1000;         % Hint: 1000

% --- Step 2: Generate Coin Flips ---
% Convert random numbers into +1 and -1:
% rand(...) > 0.5 gives 1 for Heads and 0 for Tails.
% 2 * result - 1 converts 1 to +1 and 0 to -1.
steps = 2 * (rand(num_people, num_steps) > 0.5) - 1;

% --- Step 3: Calculate Bank Account Balances ---
% cumsum(..., 2) adds results horizontally across time.
positions = cumsum(steps, 2);

% --- Step 4: Calculate Mean and Variance at Each Step ---
```

```
 mean_pos = mean(positions, 1);
 var_pos = var(positions, 0, 1);

 % --- Step 5: Print Final Results ---
 fprintf('--- Results after %d Steps ---\n', num_steps);
 fprintf('Final Sample Mean     : %.2f (Theory says 0)\n',
mean_pos(end));
 fprintf('Final Sample Variance : %.2f (Theory says %d)\n',
var_pos(end), num_steps);

 % --- Step 6: Visualize the Random Walk ---
 figure;

 % Top chart: plot the paths of the first 50 people.
 subplot(2, 1, 1);
 plot(1:num_steps, positions(1:50, :), 'Color', [0.7 0.7 0.7]);
 hold on;

 % Plot the average path using a thick red line.
 plot(1:num_steps, mean_pos, 'r', 'LineWidth', 2);
 title('Paths of 50 People & The Average Path');
 xlabel('Time (Number of Games)');
 ylabel('Profit ($)');
 grid on;

 % Bottom chart: actual variance and theoretical variance.
 subplot(2, 1, 2);
 plot(1:num_steps, var_pos, 'b', 'LineWidth', 2);
 hold on;

 % Theoretical variance of a fair random walk is Var = Time.
 plot(1:num_steps, 1:num_steps, 'k--', 'LineWidth', 2);
 title('Variance vs. Time');
 xlabel('Time (Number of Games)');
 ylabel('Variance');
 legend('Actual Sample Variance', 'Theoretical Variance', 'Location',
'northwest');
 grid on;

 % Question for report:
 % The red average line is near 0. Does this mean one player is safe?
 % Explain using the idea of variance.
```

2) Description of the code:

① Setup simulation parameters.

```
num_people = 1000;        % Hint: 1000
num_steps = 1000;         % Hint: 1000
```

② Use the rand() function to generate random numbers for all people and all steps. If the number is > 0.5, treat it as Heads. Convert these random values mathematically so that Heads becomes +1 and Tails becomes -1.

```
steps = 2 * (rand(num_people, num_steps) > 0.5) - 1;
```

③ Use the MATLAB built-in function cumsum(…, 2) (Cumulative Sum across the time dimension) to calculate the daily balances for all 1000 players.

```
positions = cumsum(steps, 2);
```

④ Use the mean(..., 1) function to calculate the average profit of the 1,000 people at each step. Then, use the var(..., 0, 1) function to calculate the variance (risk) of the 1,000 people at each step.

```
mean_pos = mean(positions, 1);
var_pos = var(positions, 0, 1);
```

⑤ Print the final results.

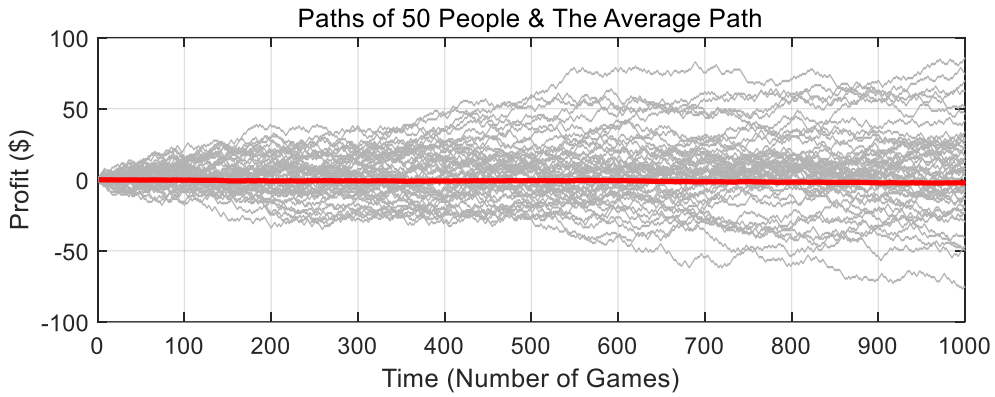
```
fprintf('--- Results after %d Steps ---\n', num_steps);
fprintf('Final Sample Mean     : %.2f (Theory says 0)\n',
mean_pos(end));
fprintf('Final Sample Variance : %.2f (Theory says %d)\n',
var_pos(end), num_steps);
```

⑥ Plot the balance paths of the first 50 people.

```
subplot(2, 1, 1);
plot(1:num_steps, positions(1:50, :), 'Color', [0.7 0.7 0.7]);
hold on;
```

⑦ Use hold on to plot the Average Path (Mean) with a thick red line.

```
plot(1:num_steps, mean_pos, 'r', 'LineWidth', 2);
title('Paths of 50 People & The Average Path');
xlabel('Time (Number of Games)');
ylabel('Profit ($)');
grid on;
```

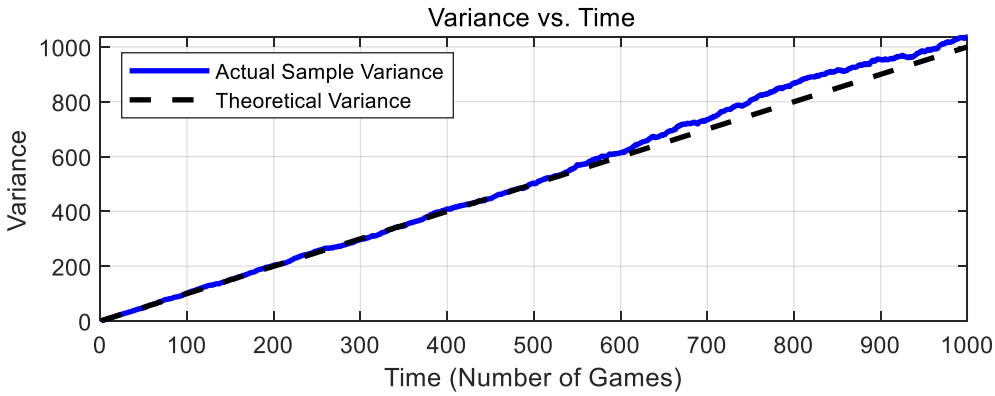


⑧ Plot the actual Variance you calculated.

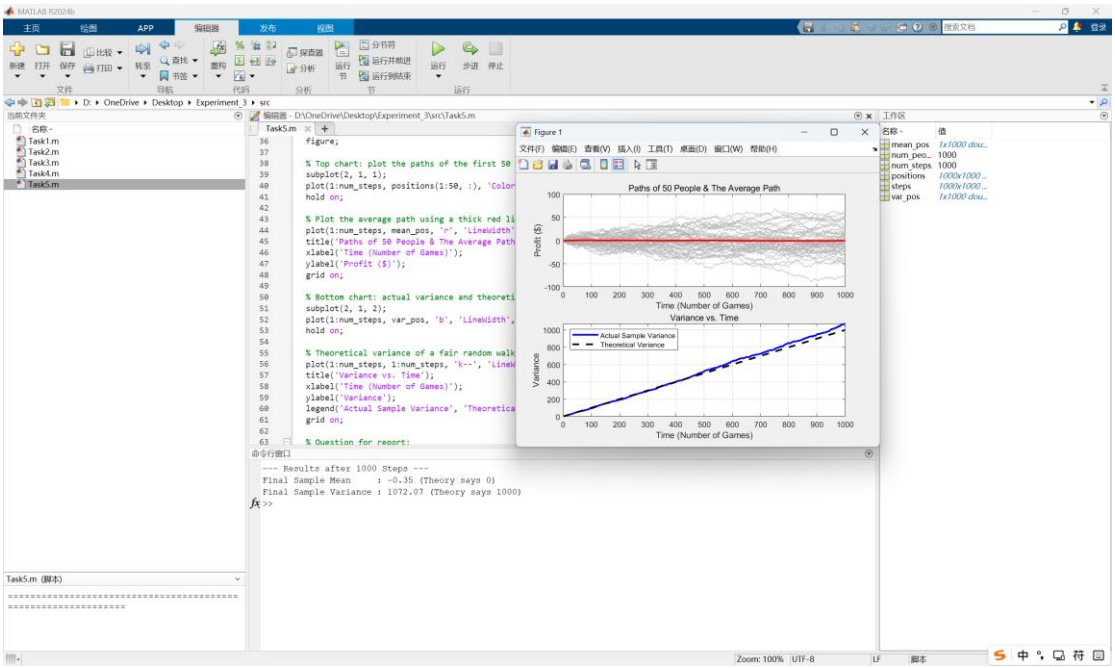
```
subplot(2, 1, 2);
plot(1:num_steps, var_pos, 'b', 'LineWidth', 2);
hold on;
```

⑨ Use hold on to plot the Theoretical Variance.

```
plot(1:num_steps, 1:num_steps, 'k--', 'LineWidth', 2);
title('Variance vs. Time');
xlabel('Time (Number of Games)');
ylabel('Variance');
legend('Actual Sample Variance', 'Theoretical Variance', 'Location',
'northwest');
grid on;
```



3) Screenshot of the terminal output:



V、 Extended Questions

- **Task 1:** Report the values of mean_math, mean_builtin, var_math, and var_builtin. Briefly state whether the manual results agree with the MATLAB built-in results.

The recorded values were mean_math = -0.0000, mean_builtin = -0.0000, var_math = 1.0036, and var_builtin = 1.0036. The manual results agree with the MATLAB built-in results. This is because the manual mean uses $\frac{sum(X)}{N}$, while the manual sample variance uses the same denominator N - 1 as MATLAB var(X).

- **Task 2:** Report the results for all 4 experiments, including sample size, number of bins, Riemann sum estimate, actual sample mean, theoretical mean, and the errors. Briefly explain how larger sample size and finer bin size affect the accuracy.

The four groups of Riemann-sum results were recorded as follows:

For N = 100000, the actual sample mean was 9.997822 and the theoretical mean was 10.000000. With 10 bins, the Riemann sum was 9.999548, the error from the sample mean was 0.001726, and the error from the theoretical mean was -0.000452. With 100 bins, the

Riemann sum was 9.997508, the error from the sample mean was -0.000314, and the error from the theoretical mean was -0.002492.

For N = 1000000, the actual sample mean was 10.000855 and the theoretical mean was 10.000000. With 10 bins, the Riemann sum was 10.000221, the error from the sample mean was -0.000633, and the error from the theoretical mean was 0.000221. With 100 bins, the Riemann sum was 10.000841, the error from the sample mean was -0.000014, and the error from the theoretical mean was 0.000841.

A larger sample size usually makes the sample mean closer to the theoretical mean because random fluctuations are averaged out. A larger number of bins gives a finer histogram and reduces the discretization error of the Riemann-sum approximation, although small differences may still appear because the data are random and the bin edges are discrete.

- **Task 3**: Report the covariance value, the theoretical mean of W, the actual mean of W, the actual variance of W, the variance from the correct formula, and the variance from the wrong formula. Briefly explain why covariance must be included.

The covariance value was Cov(X,Y) = 0.7947. The theoretical mean of W was 0.0543, and the actual mean of W was also 0.0543. The actual variance of W was 19.5743. The variance calculated by the correct formula was 19.5743, while the wrong formula without covariance gave only 10.0376. Therefore, ignoring covariance caused an error of 9.5367.

Covariance must be included because X and Y are not independent in this experiment. Since Y = 0.8X + 0.2Z, X and Y tend to move in the same direction. In the variance of a linear combination, this joint movement contributes the extra term 2abCov(X,Y). If this term is omitted, the variance is seriously underestimated.

- **Task 4:** Report the final sample mean and variance of the St. Petersburg game. Briefly describe the behavior of the cumulative curves and the "spikes" caused by rare large payoffs. **Answer the following question:** *Mathematical theory says you should be willing to pay millions of dollars to play this game because the Expected Value is*

Infinity. If a casino asks you to pay $20 to play this game just once, would you accept? Use your "Final Sample Mean" to justify your answer.

After 100000 games, the final sample mean of the St. Petersburg game was 26.40, the final sample variance was 3830560.58, and the highest prize won was 524288. The cumulative mean and cumulative variance did not change smoothly. Rare very large payoffs created obvious spikes, and after each spike the curves slowly decreased or stabilized until another rare event occurred.

If the casino asks me to pay $20 to play the game once, then based only on the final sample mean of 26.40, the average payoff in this simulation is higher than the price, so the sample-mean argument would support accepting the game. However, the extremely large variance shows that one single play is still very risky. Most plays give small payoffs, and the high average is mainly caused by rare huge prizes. Therefore, a risk-neutral decision based only on the sample mean may accept it, but a cautious player should not rely on the mean alone.

- **Task 5:** Report the Final Sample Mean and Variance of the random walk. **Answer the following question:** *Look at your top plot. The red line shows that the group's Average Profit is exactly $0. However, look at the gray lines (individual players). Does an average of $0 mean a single player is safe? Why is this game actually extremely dangerous for a single player?*

After 1000 steps, the final sample mean of the random walk was -0.35, which is close to the theoretical mean 0. The final sample variance was 1072.07, close to the theoretical variance 1000.

An average profit of 0 does not mean a single player is safe. The red average line only describes the group average, while the gray individual paths show that different players can move far above or far below zero. As time increases, the variance grows almost linearly, so the possible loss or gain of a single player becomes wider and wider. This is why the game is dangerous for one player even though the expected profit is zero.

VI、 Summary and Experiment Reflections

In this experiment, I used MATLAB simulations to study expectation, variance, covariance, the St. Petersburg paradox, and the one-dimensional random walk. Through Task 1, I confirmed that the manual formulas for sample mean and sample variance are consistent with MATLAB built-in functions when the same sample-variance denominator is used. Through Task 2, I learned that a histogram normalized as a probability density can be used to approximate expected value by a Riemann sum, and that both sample size and bin number affect the numerical accuracy. Through Task 3, I verified that the expected value of a linear combination follows the linearity property, while the variance of a linear combination must include the covariance term when variables are correlated. Tasks 4 and 5 further showed that mean alone is not enough to describe risk: the St. Petersburg game has rare large payoffs that dominate the average, and a fair random walk keeps an expected value near zero while its variance expands over time.

The main programming challenges were using element-wise operations such as .^ for arrays, correctly choosing the denominator N - 1 for sample variance, extracting the bin centers and bin widths from the histogram object, and reading the covariance value from the off-diagonal element of the covariance matrix. The mathematical challenge was understanding why two formulas that look similar can produce very different variance results when covariance is ignored. Another important observation was that random simulation results are not fixed; each run may produce slightly different numerical values, especially in heavy-tailed experiments such as the St. Petersburg game.

Overall, this experiment helped me connect probability theory with numerical simulation. Before the experiment, expectation and variance were mainly formulas in the textbook. After running the MATLAB programs and observing the figures, I understood that expectation describes the center or long-run average, while variance describes the spread and risk. In practical decision-making, especially in gambling or repeated random processes, relying only on expected value can be misleading. A complete analysis should consider both the average result and the uncertainty around that average.